# Stress Lab — Economic Model

How the layered shields keep a RoundFi pool solvent under default

RoundFi
2026-06-23

CONTENTS

1. The thesis
2. The contribution split — three streams from one installment
3. The Seed-Draw self-funding guard
4. The Triple Shield
5. The vesting mirror — discipline releases what default seizes
6. Stake economics — why the discount doesn't raise risk
7. The yield cascade — surplus without subsidy
8. Solvency invariants — the recoverability summary

> **Derived document.** This is a derivation of `docs/spec/MASTER-SPEC.md` (§4.3 Contribution split, §4.4 Stake & the Triple Shield, §4.5 Default settlement, §8 Economics). The Master Spec is the single source of truth; if a number here disagrees with it, the Master Spec wins. Every constant is pinned to the deployed Jun 2026 source ( `programs/roundfi-core/` , `crates/math/` ).

## 1. The thesis

A rotating savings pool (ROSCA) has one structural hazard: **someone draws the credit early and then stops paying.** They received the carta in cycle 2; they owe installments through cycle 8; they walk. In an informal *consórcio* that loss lands on the remaining members and the group dissolves. RoundFi's job is to make that default **recoverable** — the pool keeps paying its drawn members, the loss is absorbed by capital the defaulter themselves posted, and the event becomes a durable reputation signal rather than a dissolution.

This document is the **Stress Lab**: the solvency model and the invariants that make a default recoverable. It frames three things. First, that the pool is **self-funding** — no external subsidy is required for the cycle-0 draw to clear. Second, that a **Triple Shield** of layered capital, seized in a fixed order, covers a missed installment without touching honest members' float. Third, that the **stake discount** offered to proven members (50% → 3% of the credit) does **not** raise protocol risk, because the upper tiers are gated on a long completed -pool history that is itself the risk signal.

The whole model rests on one separation carried over from the reputation design: **receiving capital is not merit; keeping your obligations after receiving it is.** The shields collateralize that distinction; the self-funding guard makes it affordable.

## 2. The contribution split — three streams from one installment

Every `contribute` call splits the member's `installment` into three streams before any of it reaches the pool float. The split is enforced on-chain, not trusted off-chain.

| Slice | bps | Destination | Role |
|---|---|---|---|
| **Solidarity** | 100 (1%) | `solidarity_vault` | First tranche of any default cover |
| **Escrow** | `escrow_release_bps` (default 2500 = 25%) | `escrow_vault` | Vesting collateral / second tranche |
| **Pool float** | remainder (default 74%) | `pool_usdc_vault` | What pays the drawn member |

Source: MASTER-SPEC §4.3; `crates/math/src/seed_draw.rs`.

The three percentages are not arbitrary — each maps to a defensive role. The **1% solidarity** slice is a small, communal, always-first buffer (the "Cofre Solidário"). The **25% escrow** slice is the member's own money, held back and released as they prove discipline (§4). The **74% float** is the working capital that actually funds the rotating draw. Note that the solidarity and escrow slices are taken **off the top**: only 74% of each installment is available to pay the person drawn this cycle. That retention is what the next section's guard is built around.

## 3. The Seed-Draw self-funding guard

The single most important solvency invariant is that **the very first draw can be paid out of contributions alone.** In cycle 0, members have each paid one installment; one member draws the full credit (`carta`). If the float retained from those installments is smaller than the credit, the pool is insolvent on day one — the cycle-0 claim under-runs the vault.

RoundFi refuses to create such a pool. `create_pool` evaluates the **Seed-Draw viability guard** before allocating, and rejects any geometry whose math doesn't close:

```
members × installment × (MAX_BPS − solidarity_bps − escrow_release_bps) /
MAX_BPS  ≥  credit
```

With the default split this is the headline form from §8.1:

```
members × installment × 0.74  ≥  credit
```

Source: MASTER-SPEC §4.3 / §8.1; `crates/math/src/seed_draw.rs`. A pool that fails the guard reverts with **`PoolNotViable`** at creation time, so the cycle-0 claim can never trigger a **`WaterfallUnderflow`** at runtime. The check is moved to the earliest possible moment — allocation — so an unviable pool never reaches `Active`.

### 3.1 Why 74% and not 100%

A naïve ROSCA would let 100% of each installment flow to the draw, maximizing the credit a given group can support. RoundFi deliberately gives up that headroom: the 26% it diverts (1% + 25%) is the capital that makes a default recoverable. The guard therefore encodes a **conservative** self-funding condition — the pool is solvent on the float **after** the shields have already been funded, not before. The trade is legible: a slightly smaller credit per installment in exchange for a pool that survives a defaulting member.

### 3.2 What the guard does and does not promise

The guard is a **cycle-0** invariant. It guarantees the pool can fund its first draw from contributions without subsidy, and — because `cycles_total == members_target` and each slot draws exactly once (MASTER-SPEC §4.1, the hard geometry invariant) — that the steady-state float arithmetic is closed. It does **not** by itself promise solvency through an arbitrary sequence of defaults; that is the job of the Triple Shield in §4. The two work together: the guard keeps the honest pool self-funding, and the shields backfill the float when a member fails to contribute.

## 4. The Triple Shield

When a member draws and then stops paying, the missed installment leaves a hole in the float — the pool still owes its **next** drawn member a full credit. The Triple Shield is the layered capital that fills that hole, seized in a fixed order on `settle_default` so that the cheapest, most communal buffer is spent first and the defaulter's own posted capital absorbs the rest.

### 4.1 The three layers

| Layer | Capital | Funded by | Whose money |
|---|---|---|---|
| 1 | **Solidarity vault** | 1% of every installment, all cycles | The pool's |
| 2 | **Member escrow** | 25% of the defaulter's installments | The defaulter's |
| 3 | **Member stake** | Locked at join, sized by tier | The defaulter's |

The progression is intentional. Layer 1 is **communal and small** — a shared first-loss buffer that smooths the common case (a single late installment) without reaching into anyone's collateral. Layers 2 and 3 are the **defaulter's own capital**: their vested-but-unreleased escrow, then their join stake. By the time the pool touches the stake, it has already exhausted the defaulter's other on-chain balances. Honest members' float is never the source of cover.

### 4.2 The seizure order

`settle_default(cycle)` draws cover in exactly this sequence (MASTER-SPEC §4.4):

1. **Solidarity vault** — up to the missed installment.
2. **Member escrow balance** — up to the remaining shortfall.
3. **Member stake** — the remainder.

All seized USDC flows into **`pool_usdc_vault`**, so the pool can still pay its drawn member. The defaulter is marked `defaulted = true` and a `SCHEMA_DEFAULT` attestation fires (the −500 score event of MASTER-SPEC §5.3).

The order is **solidarity → escrow → stake**, never the reverse. Seizing the shared buffer first means a recoverable, one-installment miss can be covered communally and cheaply; reaching the stake — the largest and most punitive layer — is reserved for the genuine shortfall that the first two layers couldn't close.

### 4.3 Worked stress cases

The cases below are illustrative arithmetic over the §4.4 seizure order and the §4.3 split; they are not new facts, only the model applied. Let a missed installment be `I`, the solidarity vault hold `S`, the defaulter's escrow balance `E`, and their stake `K`.

| Case | Cover drawn | Touches honest float? |
|---|---|---:|
| `S ≥ I` | All from solidarity | No |
| `S < I ≤ S + E` | Solidarity, then escrow | No |
| `S + E < I ≤ S + E + K` | Solidarity, then escrow, then stake | No |
| `I > S + E + K` | All three layers, capped at total | Residual* |

*The first three rows are the designed envelope: the missed installment is fully covered by the defaulter's posted capital plus the communal buffer, and the float is made whole. The fourth row is the **residual gap** the model is honest about — if a shortfall ever exceeded the entire Triple Shield, the layers cap at what they hold. The stake-sizing ladder (§6) exists precisely to keep that case out of reach: a member who can draw a large credit at a low stake is one whose long completed-pool history is itself the assurance they will not be in row four.

## 5. The vesting mirror — discipline releases what default seizes

Seizure is one half of the escrow surface; **vesting** is the mirror. The same escrow balance that backstops a default is also the member's own collateral, returned to them in tranches **as they prove on-time discipline.**

An on-time member progressively releases their stake from escrow via `release_escrow(checkpoint)`, gated on `member.on_time_count ≥ checkpoint` (MASTER-SPEC §4.4):

- **Pay on time** ⇒ the corresponding tranche unlocks and can be released.
- **Pay late** ⇒ the stake stays locked; `release_escrow` reverts with **`EscrowLocked`**.

This is the on-chain expression of **"discipline is rewarded, delinquency is collateralized."** The two halves are economically symmetric:

| Behavior | Effect on escrow |
|---|---|
| On-time payment | Advances `on_time_count` ; unlocks a vesting tranche |
| Late payment | No advance; stake remains locked ( `EscrowLocked` ) |
| Default | Escrow then stake **seized** into the pool float |

The mirror is what makes the shield **fair as well as solvent.** A member who pays through to the end recovers their escrow and stake in full via on-time release; a member who defaults forfeits exactly that capital to the pool they failed. The collateral is never confiscated from the disciplined — it is released back to them on the same `on_time_count` signal that proves they earned it.

## 6. Stake economics — why the discount doesn't raise risk

The user-facing reward of the reputation engine is the **stake discount ladder**. A member stakes a fraction of the credit at join, sized by their tier (MASTER-SPEC §5.1):

| Tier | Name | Stake (bps of credit) | Min completed pools |
|---|---|---:|---:|
| L1 | Iniciante | 5000 (50%) | 0 |
| L2 | — | 2500 (25%) | 2 |
| L3 | Veteran | 1000 (10%) | 3 |
| L4 | Elite | 300 (3%) | 8 |

Source: MASTER-SPEC §5.1 / §8.3; `roundfi-reputation/src/constants.rs` ( `STAKE_BPS_LEVEL_*` , `LEVEL_*_MIN_CYCLES` ). The progression is **50** → **25** → **10** → **3**: an L4 Elite member frees roughly **94%** of the capital an L1 must lock.

### 6.1 The apparent paradox

Lowering the stake appears to weaken layer 3 of the Triple Shield — a defaulter at L4 forfeits only 3% of the credit, not 50%. Read naïvely, the discount looks like it trades solvency for a nicer incentive. It does not, and the reason is the gate.

### 6.2 The resolution — the discount is gated on the risk signal itself

The stake is not discounted for **claiming** a tier; it is discounted only after a member has **earned** it by completing pools to the end. The upper tiers are gated on a long completed-pool history (MASTER-SPEC §8.3):

- **L2** requires **2 completed pools** (raised from 1 in ECO-V52, so the 4× stake discount is never reachable on a single self-dealt pool).
- **L3** requires **3**.
- **L4** requires **8** — roughly a multi-year honest history.

Each `cycles_completed` increment fires only on a `POOL_COMPLETE` attestation — a pool paid **through to the end** — and carries a **30-day per-subject cooldown**

( `MIN_POOL_COMPLETE_COOLDOWN_SECS` ). The history that unlocks the discount cannot be parallelized or bought; it accrues over wall-clock time, one completed obligation at a time.

So the protocol's risk **does not rise as the stake falls**, because the very condition that lowers the stake — a long record of completed pools — is itself the evidence that the member is unlikely to default. The stake is high precisely while a member is unproven (L1, 50%) and falls only as the behavioral case against default strengthens. The discount is **priced by demonstrated reliability**, not granted on entry.

### 6.3 The identity floor on the cheapest tier

The most-discounted tier carries the strongest additional wall: **L4 Elite always requires `identity_verified`**, independent of the configurable identity gate (MASTER-SPEC §5.2). The tier that risks the least locked capital is therefore also the one bound to proof-of-personhood — closing the path where an anonymous operator manufactures a low-stake position to default on. The economic prize and the strongest gate sit on the **same** tier by construction.

## 7. The yield cascade — surplus without subsidy

The solvency model is conservative by design (§3), which means idle float accumulates. Rather than let it sit, the pool can deposit idle float into a yield adapter ( `deposit_idle_to_yield` ) and harvest the surplus ( `harvest_yield` ). The realized surplus runs a fixed waterfall (MASTER-SPEC §8.2):

| Order | Slice | Sizing | Nature |
|---|---|---|---|
| 1 | **Protocol fee** | `DEFAULT_FEE_BPS_YIELD` = 20% gross | Treasury — the only outflow |
| 2 | **Guarantee Fund** | logical earmark | `pool.guarantee_fund_balance` |
| 3 | **LP slice** | `config.lp_share_bps` (default 65%) | `pool.lp_distribution_balance` |
| 4 | **Residual** | remainder | Stays in `pool_usdc_vault` |

The order is **20% → Guarantee Fund → 65% LP → residual.** Only the protocol fee is a **physical outflow**; the Guarantee Fund and LP slices are **logical earmarks** on pool balances, and the residual stays in the float as the participants' "prêmio de paciência" (patience premium). A slippage floor ( `harvest_yield(min_realized)` ) protects against an adapter under-reporting yield.

Two points matter for the solvency model. First, the **Guarantee Fund earmark** grows a pool-level reserve from surplus the conservative float threw off — a fourth, accumulating cushion behind the Triple Shield, funded by yield rather than by diverting contributions. Second, because the fee is the **only** outflow and the rest are earmarks, harvesting never moves capital out of the pool's solvency envelope except for that single, bounded protocol fee. Adapters are pluggable: the mock serves devnet, Kamino is the mainnet target.

## 8. Solvency invariants — the recoverability summary

Pulling the model together, a RoundFi default is recoverable because a small set of invariants hold simultaneously. Each is enforced in code at the earliest possible point, not assumed.

| Invariant | Enforced by | Failure mode it removes |
|---|---|---|
| `members × installment × 0.74 ≥ credit` | `create_pool` → `PoolNotViable` | cycle-0 `WaterfallUnderflow` |
| `cycles_total == members_target` | `create_pool` (geometry invariant) | orphan cycles / unfunded draws |
| Seize solidarity → escrow → stake | `settle_default` | honest float absorbing a loss |
| On-time release only (`on_time_count`) | `release_escrow` → `EscrowLocked` | premature collateral withdrawal |
| Default seizes before profile is portable | reputation keyed by **subject = wallet** | walking away with clean credit |
| Low stake gated on completed-pool history | `LEVEL_*_MIN_CYCLES` + 30-day cooldown | cheap-tier default farming |

The chain reads cleanly: the **guard** makes the honest pool self-funding; the **Triple Shield**, seized in order, covers the defaulter's miss from the defaulter's own capital plus a communal buffer; the **vesting mirror** returns that same capital to members who stay disciplined; and the **gated discount** lowers the stake only as the behavioral case against default strengthens. No layer relies on trust where it could rely on an on-chain invariant — which is exactly the property a grant reviewer or actuarial reader needs to see before believing "recoverable" rather than merely hearing it.

---

*Cross-references: protocol mechanics →* `02-technical-whitepaper` *; program topology →* `03-architecture-spec` *; the reputation ladder that prices the stake →* `04-behavioral-reputation-score` *; adversarial economics →* `09-risk-and-compliance` *. Full economics live in MASTER-SPEC §4 and §8.*